Masterhub Database Integration Report

Implementation status and DevOps handoff

14 October 2026 | Version 1.0 | Prepared for the Masterhub development and DevOps teams

Executive summary

Masterhub now uses PostgreSQL for application records and operational history. The active database was migrated from local Windows PostgreSQL to Neon. Masterhub itself continues to run on Windows; hosting its database in the cloud has not deployed the application or its dashboard to a cloud server.

The completed work replaces SQLite as the runtime database, removes the Supabase runtime integration, preserves existing device identities, and records new commands, connection events, acknowledgements and errors. A Saved Activity dashboard panel reads persisted history from PostgreSQL. A subsequent fix reconciles deleted MQTT device identities with the dashboard registration list.

Delivered capabilities

- One PostgreSQL connection for accounts, sessions, phone ownership, device registrations and saved runtime events.
- A durable local event queue with background cloud writes, retry after failure and duplicate-safe replay of queued events.
- Verified transfer of six tables to Neon, retaining the original local database and a fresh backup.
- Device registration refresh, event filters, pagination and visible storage health in the dashboard.

Verified outcome and scope

At the cloud cutover on 13 October 2026, 19 MQTT device identities, one registry record and 383 runtime events were transferred. Content hashes and row counts matched across all six tables. Ten subsequent events were confirmed in Neon. These are historical acceptance results, not live totals for the report date.

Full cloud application hosting, Linux and Raspberry Pi compatibility, dashboard access for friends, and production authentication remain separate work. This report describes the implemented database integration and the remaining operational requirements.

Report navigation

2 System design and database schema
3 Event recording and dashboard behavior
4 Migration and verification evidence
5 Registration deletion and troubleshooting
6 Configuration and DevOps deployment
7 Operations limitations and source references

System design and database schema

Devices communicate with the running Masterhub backend through MQTT or local interfaces. Masterhub maintains live control state in memory, writes durable application records to PostgreSQL, and records operational events through a local outbox. The browser calls Masterhub APIs; it does not connect directly to Neon or receive database credentials.

Location and responsibility

- Windows host: Flask application, device integrations, MQTT clients, live state and pending event files.
- Neon PostgreSQL: persistent application tables and saved runtime history.
- pgAdmin: an administration and viewing client for either local PostgreSQL or Neon; it is not the database storage engine.
- Local recovery assets: the former SQLite file, the preserved local PostgreSQL database and migration backups. They do not automatically mirror new cloud writes.

Application tables

- users: identity-generated user ID, unique username, password hash and creation time. Passwords are hashed rather than stored as login plaintext.
- sessions: hashed session token, user reference and expiry time. Existing mobile-account authentication uses these records.
- mobile_devices: device ID, owning user, installation ID and display name. A user and installation pair is unique.
- mqtt_devices: unique device ID and username, platform type and update time. This is the identity source for MQTT-discovered phones.
- device_registry: a single row with ID 1 containing a JSON document stored as text. It holds dashboard targets, capabilities, transport details and topic prefixes.
- runtime_events: append-oriented operational history with an identity sequence, unique event ID, installation ID, receipt time, event kind, device ID, status, optional command ID, topic and JSONB payload.

Database behavior

The psycopg driver uses parameterized SQL and dictionary rows. Successful operations commit; failures roll back and connections close. Schema creation uses a transaction-level advisory lock and is cached within each process. Time fields for account and event records use millisecond values that require BIGINT storage.

Indexes support event queries by installation and time, device and time, and command ID. History uses the sequence column for pagination. The installation ID scopes event history only; it does not provide user authorization or isolate all application tables for independent tenants.

Event recording and dashboard behavior

What Masterhub records

Hardware MQTT captures broker connections and disconnections, device status, acknowledgements, errors and sensor messages. Outgoing commands are recorded as published or failed. Mobile MQTT records connection changes, status and heartbeats, media messages, acknowledgements, command publication and timeouts observed by the existing status checks.

USB integration records connections, command requests and results, and parsed incoming JSON messages. The command engine records requests and returned results. Dashboard activity and Cortex connection changes are also recorded. Invalid packets rejected before a recording hook and historical in-memory events are not retrospectively reconstructed.

Reliable event transfer

An event first becomes a JSON file in data/event-outbox. The file is written, flushed and atomically renamed before a background writer reads it. The writer sends batches of up to 100 events and removes queue files only after the PostgreSQL transaction commits. The normal retry wait is five seconds; connection and query time can extend the effective interval.

A unique event_id and ON CONFLICT handling prevent the same queued event from being inserted twice after restart or uncertain cleanup. This does not deduplicate separate deliveries of the same MQTT packet: each newly recorded delivery receives its own event ID. Malformed queued JSON is quarantined as an .invalid file for review.

Event capture performs local disk work but does not wait for a cloud database connection. Account and registration operations still use synchronous PostgreSQL transactions and can fail or wait during an outage. The queue is not an offline fallback for every application function.

Acknowledgement meaning

A published command proves transport submission, not device execution. Command IDs are retained where supplied. Uncorrelated acknowledgements are marked UNCONFIRMED rather than assigned to an arbitrary command. Retained messages are identified in the event payload. Late acknowledgements can be saved after restart even if the original command is no longer in memory.

Dashboard display

Saved Activity reads GET /api/database/activity, normally refreshing every five seconds. It provides device and event filters, older-record pagination and expandable payloads. The API accepts a limit from 1 to 200 and an optional sequence cursor. It returns a generic HTTP 503 response if history cannot be read.

Saved history is separate from live presence. Heartbeat freshness and current transport state continue to drive live indicators. Opening a saved event or browsing older records pauses automatic refresh so the user can read it. Clearing the original live Activity Log does not delete database history.

Migration and verification evidence

From SQLite and Supabase to PostgreSQL

The original SQLite store contained accounts, sessions and device registrations; Supabase mirrored MQTT identities. Runtime storage was changed to PostgreSQL, with PostgreSQL SQL parameters, identity columns, BIGINT timestamps and driver-specific error handling. The Supabase background sync and active connection settings were removed. The hosted Supabase project itself was not deleted.

The one-time migration tool reads SQLite without changing it and creates a consistent backup. Optional Supabase import preserves cloud-only identities, checks conflicts and imports into an empty destination. It verifies counts and resets the user ID sequence. Migration-only Supabase code remains available for historical imports; it is not used by the running application.

From local PostgreSQL to Neon

The cloud migration used an empty PostgreSQL 18 target and the direct TLS connection. Masterhub was stopped, pending events were drained locally, and a fresh custom-format backup was created. The restore ran in a single transaction without importing local role ownership or privilege assignments. Row counts and SHA-256 content hashes were compared before DATABASE_URL was switched.

The saved verification report records: users 0; sessions 0; mobile_devices 0; device_registry 1; mqtt_devices 19; runtime_events 383. The empty account tables are part of the verified result, not missing data. Existing local data was preserved as a recovery source.

After restart, the dashboard and saved-history API returned HTTP 200. Ten new events were confirmed in the cloud; the final cutover check reported no pending events, failed queue writes or storage error. Verification applies to that check, not an ongoing availability guarantee.

Test evidence from implementation

- Initial PostgreSQL account, device, transfer and MQTT integration selection: 20 passed. Migration cleanup recheck: 3 passed.
- Event persistence plus account and registration selection: 24 passed.
- Expanded event storage, dashboard rendering, mobile, hardware and MQTT integration selection: 42 passed.
- Registration deletion refresh and registration regressions: 18 passed.

These selections overlap and must not be added together as a unique test total. Tests covered rollback, retries, replay, filtering, payload redaction, retained messages, uncorrelated acknowledgements, registry deletion and existing protocol behavior. They used isolated PostgreSQL test schemas; the report does not claim load testing or a production security audit.

Preserved evidence

The migration backup is data/masterhub-before-cloud-20261013T143029Z.backup. Its companion .verification.json file contains the verified counts and content hashes. The older masterhub_cloud.backup predates runtime event storage and should not be used as the latest full recovery snapshot.

Registration deletion and troubleshooting

Why deleted registrations remained visible

Deleting rows from mqtt_devices initially left the device list unchanged because dashboard target details also lived in device_registry and in process memory. The investigation found two surviving identity rows while 17 deleted phone identities still appeared in the registry. Changing one store alone did not invalidate the others.

The fix checks MQTT-discovered mobile targets against the identity table, refreshes the registry on device-list access with a three-second cache interval, and filters deleted identities before saving. If a removed device was active, the target resets to Local Host. Manual targets and the built-in Local Host are preserved. Application unregister also deletes the corresponding MQTT identity.

Live verification and process correction

The first restart check still showed 24 targets. Investigation found two Masterhub processes listening on port 5000; an old process continued serving stale data. Both known copies were stopped and one updated instance was started. The final check showed 8 targets, including 3 mobile devices, Local Host as the active target, and a working saved-history API.

One phone registered again during verification. Deleting a registration is not a permanent access block: a connected device can submit a new registration. Future blocking requires an explicit policy or denylist. The refresh behavior depends on database availability and the next dashboard poll; it is not instantaneous push synchronization.

Viewing cloud records in pgAdmin

Select the cloud server connection, open the configured database, then expand Schemas, public and Tables. Use View/Edit Data on runtime_events for operational history or mqtt_devices for phone identities. The cloud database name can differ from the local name masterhub; use the actual configured database.

```
SELECT to_timestamp(received_at / 1000.0) AS event_time,
       kind, device_id, status, command_id
FROM public.runtime_events
ORDER BY sequence DESC
LIMIT 100;
```

Common checks

- History appears empty: confirm the cloud connection, preserve MASTERHUB_INSTANCE_ID, generate new activity and inspect queue health.
- Deleted targets persist: confirm edits were saved in the cloud database, check the identity versus registry distinction, and ensure only one application process is serving the dashboard.
- Connection fails: check DNS, network reachability, TLS, role credentials and schema permissions. An installed driver alone does not prove database access.
- Queue grows: inspect cloud connectivity, disk capacity and quarantined files. Do not delete pending files to clear the warning.

Configuration and DevOps deployment

Connection settings

DATABASE_URL selects the active PostgreSQL connection and overrides the separate PG settings. CLOUD_DATABASE_URL is a staging field for the migration tool; setting it alone does not switch the application. Keep passwords and complete connection URLs in the server environment or a secret manager, never in browser code or this report.

```
DATABASE_URL='postgresql://USER:PASSWORD@HOST:5432/DB?sslmode=require'
MASTERHUB_INSTANCE_ID=masterhub-main
# Optional persistent queue location
MASTERHUB_EVENT_SPOOL=data/event-outbox
```

The example contains placeholders. URL-encode credentials where needed and retain the provider-required TLS and channel-binding options. The PGHOST, PGPORT, PGDATABASE, PGUSER and PGPASSWORD fields support a separate-field configuration. PGSSLMODE supplies TLS mode for that configuration. Existing process environment values take precedence over values loaded from .env.

DevOps handoff sequence

- Provision a PostgreSQL 18 or newer destination for the existing PostgreSQL 18 backup workflow. Supply the application role, TLS settings, permitted network path and backup policy.
- Review database/schema.sql. The current startup path creates tables and indexes, so schema CREATE permission is required. A separately privileged migration role and versioned migrations are future hardening work.
- Stop the source application and other writers. Preserve its instance ID and pending event files. Take a fresh backup and verify the destination is empty before using tools/migrate_cloud.py.
- Verify every table, then change DATABASE_URL. Run python tools/check_postgresql.py and restart exactly one application instance.
- Check dashboard history, registration behavior and a newly saved event. Confirm queue health. Preserve the old database and backup until acceptance is complete.

Application hosting remains separate

The completed integration hosts the database on Neon while the application runs on Windows. A complete server deployment must configure a production application server, HTTPS, route authentication and authorization, process supervision, secret storage and persistent queue storage. It must also review MQTT authentication and transport security independently of PostgreSQL TLS.

Linux or Raspberry Pi deployment requires adapting Windows paths and desktop automation dependencies. Network MQTT devices can communicate with a remote backend; a cloud server cannot directly operate a USB device attached to a distant computer. Local hardware functions require a network-capable device or an appropriate device-side agent.

Multiple workers or installations must not be enabled merely by sharing DATABASE_URL. Registration storage and live control state are not designed as a fully coordinated multi-instance service. The single registry document also makes concurrent whole-registry writes a remaining design constraint.

Operations limitations and source references

Backup retention and rollback

The preserved local PostgreSQL database is a pre-cutover snapshot and is no longer updated by normal cloud writes. After new data has been written to Neon, reverting to local storage requires reconciliation or a new transfer; clearing DATABASE_URL alone can expose older records. Back up the active cloud database and test restoration before treating backup availability as recovery readiness.

No automatic event retention policy is implemented. DevOps must choose retention periods, monitor cloud and queue disk growth, and define recovery time and recovery point objectives. Keep pending outbox files on persistent storage and preserve the instance ID across moves. The old SQLite masterhub.db file is a historical source, not the active database.

Known limits

- The outbox protects recorded events after successful local queuing; disk failure, oversized events or termination before the atomic rename can still prevent capture. It does not provide exactly-once device delivery.
- The queue redacts common secret fields and embedded URL credentials. Arbitrary free-form payload text may still contain sensitive data; redaction is not a complete data-loss prevention system.
- Invalid JSON is quarantined, but other database or payload failures can delay a whole batch. Acknowledgement timeouts are recorded when existing mobile status checks observe them, not by a new independent scheduler.
- The dashboard and control APIs are not yet demonstrated as safe for public multi-user hosting. Account storage does not automatically protect every route. Friend access and permanent device blocking remain future features.

Implementation references

All paths below are relative to the Masterhub project root and identify the local implementation evidence used for this report.

- core/account_store.py and database/schema.sql: connection handling, schema definitions and application tables.
- services/event_store.py and api/history.py: event queue, cloud writer, storage health and persisted-history queries.
- services/mqtt_service.py, services/mobile_mqtt_service.py, services/usb_service.py, core/engine.py and cortex/dashboard.py: event recording hooks.
- core/devices.py and api/routes.py: registry reconciliation, registration and deletion APIs.
- static/database-history.js and templates/dashboard.html: Saved Activity rendering and refresh behavior.
- tools/check_postgresql.py, tools/migrate_to_postgresql.py and tools/migrate_cloud.py: validation and migration utilities.
- tests/test_event_storage.py and tests/test_registry_refresh.py: focused persistence and synchronization regression coverage.

Operational guides: docs/POSTGRESQL_WINDOWS_SETUP.md, docs/POSTGRESQL_EVENT_STORAGE.md and docs/CLOUD_DATABASE_HANDOFF.md. Verification results are historical acceptance checks from 13 October 2026.